

Experimental Results

Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 - Machine Learning | Assignment 1

Department of Computer Science and Engineering

1. Overview

This document presents the experimental results from running the Safe Semantic Planner on 6 test cases. Each test case evaluates a different aspect of the planner: basic reachability, bad state avoidance, safety margin analysis, dynamic transition handling, goal updates, and transition additions. All tests were executed using Python 3 on a standard machine.

2. Test Case Results

Test Case 1: Basic Reachability

Graph: S -> A -> B -> G (simple linear path, no bad states)

Metric	Value
Success	True
State Path	[0, 1, 2, 3]
Transition Path	[10, 11, 12]
Total Cost	3.0
Safety Score	inf (no bad states)
Reliability	3.0
Objective Score	210.0
Explored States	4
Planning Time	~0.000156 s

Result: PASS. The planner found the unique valid path as expected.

Test Case 2: Bad State Avoidance

Graph: Path 1: S -> A -> X -> G (X is bad), Path 2: S -> C -> D -> G (safe). The planner must avoid Path 1 and choose Path 2.

Metric	Value
Success	True
State Path	[0, 3, 5, 4]
Transition Path	[23, 24, 25]
Total Cost	4.5
Safety Score	1.4142
Reliability	3.0
Objective Score	11.3284

Explored States	5
Bad States Visited	0
Planning Time	~0.000123 s

Result: PASS. The planner correctly avoided bad state X and chose the safe detour.

Test Case 3: Safety Margin

Two valid paths exist. Path 1 (S -> A -> B -> G) is cheaper (cost=3.0) but passes close to a bad state. Path 2 (S -> C -> D -> G) costs more (cost=6.0) but stays far away.

Metric	Path 1 (Chosen)	Path 2 (Alternative)
Cost	3.0	6.0
Safety Distance	0.5	~3.16
Reliability	2.7	3.0
Objective Score	10.7	7.32

Result: PASS. A* minimizes cost, so it picks Path 1 (cost=3.0). However, the safety score of 0.5 reveals that this path passes very close to the bad state. Path 2 would have a safety distance of approximately 3.16 but costs twice as much. The scoring function captures this tradeoff. To prioritize safety, the gamma weight could be increased.

Test Case 4: Dynamic Transition

Initially S -> A -> G is available (cost=2.0). Then transition A -> G becomes unavailable, forcing the planner to find an alternative.

Metric	Before	After Replan
Success	True	True
State Path	[0, 1, 4]	[0, 2, 3, 4]
Total Cost	2.0	4.5
Explored States	3	5
Planning Time	~0.000044 s	~0.000070 s

Result: PASS. After the direct path was blocked, the planner successfully found the backup path through states B and C. Replanning time was approximately 0.00007 seconds.

Test Case 5: Goal Update

Original goal is G1 (state 2). After initial planning, the goal changes to G2 (state 4).

Metric	Goal = G1	Goal = G2 (Replan)
Success	True	True
State Path	[0, 1, 2]	[0, 1, 3, 4]

Total Cost	2.0	3.0
Explored States	3	4
Planning Time	~0.000022 s	~0.000025 s

Result: PASS. The planner successfully computed a new path to the updated goal G2. Replanning time was approximately 0.000025 seconds.

Test Case 6: Transition Addition

Initially, only the long path S -> A -> B -> G exists (cost=6.0). A new shortcut transition S -> G (cost=1.0) is then added.

Metric	Before Shortcut	After Shortcut
Success	True	True
State Path	[0, 1, 2, 3]	[0, 3]
Total Cost	6.0	1.0
Explored States	4	2
Planning Time	~0.000017 s	~0.000032 s

Result: PASS. The planner discovered the new shortcut and produced a much cheaper path (cost reduced from 6.0 to 1.0). The number of explored states also dropped from 4 to 2.

3. Summary of Results

Test Case	Goal Reached	Bad States Visited	Result
1. Basic Reachability	Yes	0	PASS
2. Bad State Avoidance	Yes	0	PASS
3. Safety Margin	Yes	0	PASS
4. Dynamic Transition	Yes	0	PASS
5. Goal Update	Yes	0	PASS
6. Transition Addition	Yes	0	PASS

4. Evaluation Metrics Summary

The following metrics were evaluated across all test cases:

- Goal success rate: 100% (6/6 tests passed)
- Bad states visited: 0 in all tests
- Planning time: All tests completed in under 0.001 seconds
- Replanning time: All dynamic replanning completed in under 0.0001 seconds

- Memory usage: Minimal (dictionary-based structures, no large matrices)
- Explored states: Ranges from 2 to 5 depending on graph complexity